Roteiro do vídeo

Sistemas Operacionais - Atividade 3 - UFS 2026.2

Duração-alvo: 15 minutos como proposta de gravação; o enunciado não fixa uma duração máxima para o vídeo.

Texto em FALA é a sugestão de locução. EM TELA orienta o operador. Tempos são estimativas para ensaio, não medições de uma gravação. Todos devem compreender sua parte. A divisão de falas não comprova contribuição técnica efetiva.

Sequência dos 11 slides

Slide e assunto	Quem fala	Janela
01. Virtualização, namespaces e cgroups	Bruno	00:00-00:50
02. Ambiente e kernel compartilhado	Iasmin	00:50-02:10
03. Evidências de namespaces	Pedro	02:10-03:35
04. Carga determinística e protocolo	Christian	03:35-05:00
05. Limites efetivos do cgroup	Marcos	05:00-06:20
06. Seis medições de tempo decorrido	Sibele	06:20-07:50
07. Throttling e escopos de memória	Marcos + Sibele	07:50-09:00
08. Mount read-only e menor privilégio	Silas	09:00-10:25
09. Evidência selecionada	Christian	10:25-12:55
10. Limitações e relação com inferência	Iasmin + Silas	12:55-14:15
11. Equipe e revisão final	Bruno	14:15-15:00

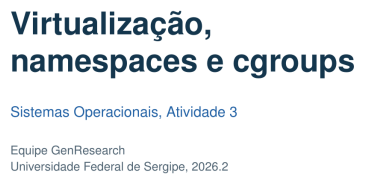
Participação por integrante

Integrante	Slides	Tempo-alvo
Bruno Amancio Ferreira	1, 11	01:35
Christian Will Silva Santos Nunes	4, 9	03:55
Iasmin Victoria Teixeira Barreto	2, 10	02:00
Marcos Vinícius Dantas Aguiar	5, 7	01:55
Pedro César Figueiredo Carneiro	3	01:25
Sibele Oliveira Cruz	6, 7	02:05
Silas Santos da Silva	8, 10	02:05

Base: apresentacao_atividade3.pdf, 11 slides em LaTeX Beamer. O slide 11 contém uma distribuição simulada de contribuições e pendências; conferir com o trabalho efetivo antes da versão final. As instruções de operação e preparação aparecem no final deste PDF.

Slide 01 - Virtualização, namespaces e cgroups

Janela 00:00-00:50 | Duração-alvo 00:50



EM TELA - não ler em voz alta

Abrir com Bruno visível e a capa compartilhada. Inserir seu nome completo na identificação da gravação.

Próxima ação

Cortar para Iasmin e exibir o slide 2.

FALA - Bruno Amancio Ferreira (00:00-00:50)

Olá, eu sou Bruno, da equipe GenResearch. Este vídeo apresenta nosso laboratório da Atividade 3 de Sistemas Operacionais. A pergunta é como namespaces mudam a visão de processos, montagens e rede, e como cgroups limitam recursos dentro de um kernel compartilhado.

Reaproveitamos a experiência da atividade anterior com Linux e medições, mas desenvolvemos os scripts deste laboratório sem o kit de apoio. Vamos acompanhar os onze slides, explicar as seis execuções e mostrar uma auditoria dos dados reais. Cada integrante vai explicar uma parte do laboratório. Iasmin vai apresentar as camadas do ambiente utilizado.

Slide 02 - Ambiente e kernel compartilhado

Janela 00:50-02:10 | Duração-alvo 01:20



EM TELA - não ler em voz alta

Exibir Iasmin identificada e o diagrama. Usar o cursor para destacar cada camada. Não apresentar o Ubuntu como host do daemon.

Próxima ação

Passar para Pedro e para o slide 3.

FALA - Iasmin Victoria Teixeira Barreto (00:50-02:10)

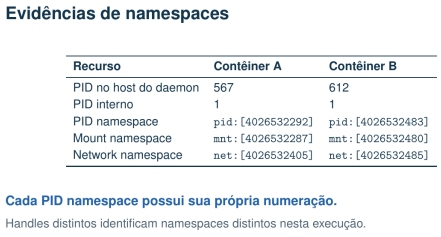
Eu sou Iasmin e vou explicar o ambiente experimental. Na parte superior do diagrama está o computador físico com Windows 10. A infraestrutura WSL2 oferece uma camada Linux intermediária. Nessa infraestrutura executa o Docker Desktop, que cria os contêineres A e B.

Os dois contêineres compartilham o kernel Linux. A imagem usada fornece Python e o espaço de usuário Debian; ela não inicia um kernel independente. Essa distinção ajuda a separar contêiner de máquina virtual: uma VM envolve um kernel convidado, enquanto aqui os processos dos contêineres usam o kernel Linux compartilhado.

Registramos doze CPUs lógicas e cerca de 3,72 gibibytes de RAM anunciados pelo daemon, além do Docker Engine e do cgroup versão dois. Também fixamos a imagem por digest para manter a mesma referência nas execuções. O Ubuntu do WSL foi usado para analisar os dados, e não deve ser confundido com o ambiente efetivo do daemon. Pedro apresenta agora as evidências de namespaces.

Slide 03 - Evidências de namespaces

Janela 02:10-03:35 | Duração-alvo 01:25



EM TELA - não ler em voz alta

Manter a tabela legível em tela inteira. Destacar os valores com o cursor, sem ler todos os dígitos dos handles.

Próxima ação

Passar para Christian no slide 4.

3/11

FALA - Pedro César Figueiredo Carneiro (02:10-03:35)

Eu sou Pedro e vou interpretar os namespaces. Os dados desta tabela foram coletados com os dois contêineres ativos e usando a mesma imagem. O Docker informou os processos 567 e 612 no host do daemon. Mesmo assim, cada contêiner mostrou seu processo inicial com PID um.

Não se trata do mesmo processo. Cada PID namespace apresenta sua própria numeração. O identificador interno precisa ser interpretado junto com a visão em que ele aparece. Também comparamos os handles de PID, mount e network. Em cada uma dessas três linhas, A e B têm identificadores distintos.

O PID namespace está ligado à visão dos processos. O mount namespace organiza a visão das montagens, e o network namespace organiza recursos de rede. O que observamos foi a separação desses namespaces nesta execução; os números podem mudar em outra execução.

Esses resultados não demonstram uma barreira de segurança absoluta. Usamos rede desativada e não testamos comunicação. Além disso, o PID informado por inspect pertence ao host do daemon, e não deve ser procurado em qualquer visão de proc sem confirmar sua correspondência. Agora Christian apresenta a carga determinística.

Slide 04 - Carga determinística e protocolo

Janela 03:35-05:00 | Duração-alvo 01:25



EM TELA - não ler em voz alta

Exibir Christian identificado. Apontar a expressão e depois a sequência. O código workload.py pode ser aberto apenas se já estiver preparado.

Próxima ação

Passar para Marcos no slide 5.

4/11

FALA - Christian Will Silva Santos Nunes (03:35-05:00)

Eu sou Christian e vou explicar o desenho da carga. Na atividade anterior havia uma espera simulada para representar geração. Para observar quota de CPU, usamos aqui um cálculo aritmético de uma thread, com memória constante e trabalho conhecido.

O laço executa vinte milhões de iterações. Em cada uma, acumula a expressão mostrada no slide e mantém o resultado em trinta e dois bits. A correção não depende apenas de repetir o laço: uma fórmula fechada, baseada nas somas dos inteiros e dos quadrados, calcula o valor esperado. As seis execuções concordaram com esse checksum.

Antes das medições principais, fizemos um piloto de um milhão de iterações e escolhemos uma quantidade segura. Depois disso, o trabalho permaneceu congelado. A ordem foi C1, C2, repetida três vezes, sempre com contêiner novo e a mesma imagem por digest.

O tempo do piloto fica separado das estatísticas. Também limitamos a duração com watchdogs de sessenta segundos dentro da carga e noventa segundos no comando externo. Marcos vai mostrar como confirmamos os limites de recursos.

Slide 05 - Limites efetivos do cgroup

Janela 05:00-06:20 | Duração-alvo 01:20



EM TELA - não ler em voz alta

Exibir Marcos identificado. Usar a tabela como guia e destacar que as linhas são leituras efetivas do kernel.

Próxima ação

Passar para Sibele no slide 6.

5/11

FALA - Marcos Vinícius Dantas Aguiar (05:00-06:20)

Eu sou Marcos e vou explicar os cgroups. Namespaces organizam visões de recursos. Já os cgroups permitem organizar processos e controlar o consumo de recursos. Neste laboratório, usamos cgroup versão dois.

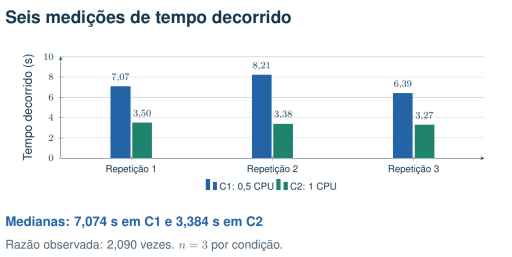
C1 recebeu quota de meia CPU e C2, de uma CPU. O arquivo cpu ponto max contém a quota e o período em microssegundos. Em C1, os valores cinquenta mil e cem mil representam cinquenta milissegundos de CPU em cada período de cem milissegundos. Esse é um teto de tempo agregado, sem reservar fisicamente meio núcleo ou fixar o processo em um núcleo.

Apenas solicitar flags ao Docker não seria evidência suficiente. Por isso, lemos os limites antes e depois do trabalho. Mantivemos a mesma memória de duzentos e cinquenta e seis mebibytes e confirmamos swap adicional zero e sessenta e quatro tarefas, incluindo threads.

Não foi necessário provocar falta de memória ou criar tarefas em excesso. A comprovação aqui é dos limites configurados e observados. Agora Sibele apresenta as seis medições.

Slide 06 - Seis medições de tempo decorrido

Janela 06:20-07:50 | Duração-alvo 01:30



Medianas: 7,074 s em C1 e 3,384 s em C2

Razão observada: 2,090 vezes. $n = 3$ por condição.

6/11

EM TELA - não ler em voz alta

Exibir Sibele identificada e ampliar o gráfico. Deixar os três pares visíveis. Os rótulos estão arredondados a duas casas; as medianas abaixo têm três.

Próxima ação

Passar para Marcos no slide 7 e manter o mesmo slide durante a fala seguinte de Sibele.

FALA - Sibele Oliveira Cruz (06:20-07:50)

Eu sou Sibele e vou interpretar os resultados. O gráfico mostra as seis observações, com três pares de barras. Azul corresponde a C1, com meia CPU, e verde a C2, com uma CPU. Todas as execuções concluíram vinte milhões de iterações com o mesmo checksum.

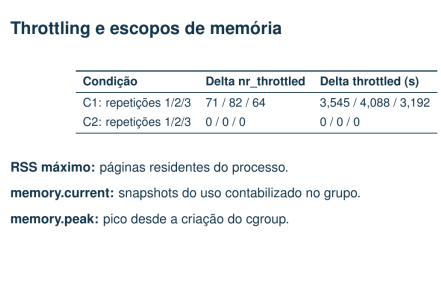
O eixo vertical mostra o tempo decorrido em segundos. Esse tempo cobre o laço de trabalho e exclui a criação do contêiner, a inicialização do Python e a coleta posterior. Assim, ele tem um escopo definido e não representa toda a duração do comando Docker.

A mediana de C1 foi cerca de sete vírgula zero sete quatro segundos. Em C2, foi três vírgula três oito quatro. A razão entre as medianas foi aproximadamente dois vírgula zero nove. Na tabela consolidada também registramos trabalho por segundo, que significa iterações por segundo e não tokens de um modelo.

C1 variou mais que C2. Como temos apenas três repetições e o host é compartilhado, o resultado deve ser interpretado dentro dessas condições. Não isolamos frequência, temperatura ou carga externa. Os contadores que Marcos apresenta a seguir ajudam a sustentar a interpretação da quota.

Slide 07 - Throttling e escopos de memória

Janela 07:50-09:00 | Duração-alvo 01:10



EM TELA - não ler em voz alta

Marcos aparece no início; fazer uma troca clara de locutor para Sibele, com identificação na tela. Nenhuma troca de slide entre as falas.

Próxima ação

Passar para Silas e para o slide 8.

7/11

FALA - Marcos Vinícius Dantas Aguiar (07:50-08:25)

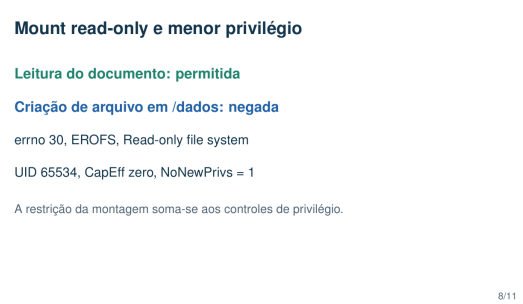
Para verificar throttling, usamos diferenças entre os contadores lidos antes e depois da carga. Nas três execuções de C1, houve aumento do número de períodos com throttling e do tempo associado à restrição. Em C2, esses deltas foram zero nessa janela. Não estamos afirmando ausência de toda restrição externa: estamos descrevendo esses contadores locais. A combinação entre limites, tempos e deltas torna a interpretação mais sustentada. Sibele continua com a memória.

FALA - Sibele Oliveira Cruz (08:25-09:00)

Na memória, distinguimos três escopos. RSS máximo vem do processo e descreve suas páginas residentes. Memory current é a contabilização do grupo nos instantes observados. Memory peak cobre a vida do cgroup, inclusive a inicialização. Por isso, os valores podem diferir e não devem ser chamados de RAM média. Também não representam uma quota de memória de GPU. Não provocamos OOM nem esgotamento de tarefas. Silas mostra agora o teste de montagem somente leitura.

Slide 08 - Mount read-only e menor privilégio

Janela 09:00-10:25 | Duração-alvo 01:25



EM TELA - não ler em voz alta

Exibir Silas identificado. Apontar o erro e, se necessário, mostrar o arquivo mount-readonly.json já aberto. A demonstração é a evidência da execução registrada.

Próxima ação

Passar para Christian no slide 9 e preparar a captura do terminal.

FALA - Silas Santos da Silva (09:00-10:25)

Eu sou Silas e vou apresentar o teste de montagem somente leitura. Criamos um arquivo de texto sintético e montamos apenas seu diretório em barra dados. A leitura funcionou. Quando tentamos criar um novo arquivo nesse mesmo ponto de montagem, o sistema negou a operação com errno trinta, EROFS, ou sistema de arquivos somente leitura.

Registramos o erro e a opção ro no mount observado. Essa combinação relaciona a negação ao caráter somente leitura da montagem. Também executamos o teste com UID sessenta e cinco mil quinhentos e trinta e quatro, sem root, com capabilities efetivas zeradas e NoNewPrivs igual a um.

É importante separar os papéis. O mount namespace organiza a visão das montagens; a opção ro restringe escrita; identidade, permissões e políticas de privilégio regulam o acesso. O conjunto reduz as ações disponíveis, mas não elimina riscos do kernel e do runtime compartilhados.

Não montamos diretórios sigilosos, a raiz do host ou o Docker socket. O teste negativo ficou dentro do diretório sintético autorizado. Christian vai mostrar a verificação dos registros guardados.

Slide 09 - Evidência selecionada

Janela 10:25-12:55 | Duração-alvo 02:30



EM TELA - não ler em voz alta

Seguir a página de operação: 0-20 s no slide; 20-55 s executar auditor e explicar a saída; 55-95 s mostrar C1-1.json; 95-125 s mostrar mount-readonly.json; 125-150 s retornar ao slide e concluir. Reservar trechos silenciosos para leitura dos campos.

Próxima ação

9/11 Retornar aos slides, avançar para o 10 e passar a palavra a Iasmin.

FALA - Christian Will Silva Santos Nunes (10:25-12:55)

Nesta demonstração, vamos conferir o lote que produziu os resultados. O comando do slide executa uma auditoria dos arquivos existentes. Não estamos criando uma nova medição nem repetindo agora as seis cargas.

A saída indica um resultado válido, seis execuções principais e sessenta e oito arquivos verificados. Isso significa que a auditoria conferiu a integridade interna, a correção do trabalho, os limites observados e outros critérios do lote.

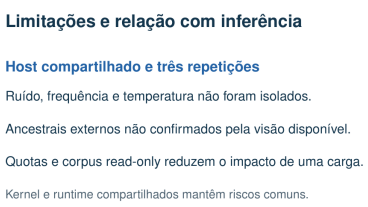
Vou abrir o registro C1 um. Aqui aparecem a condição, a quantidade fixa de trabalho, o checksum e os snapshots do cgroup. Em cpu ponto max, vemos cinquenta mil por cem mil. Nos deltas de CPU, podemos identificar o aumento dos contadores de throttling.

Agora mostramos o registro do mount somente leitura. Ele preserva a leitura bem-sucedida, o erro trinta na tentativa de escrita e os controles de privilégio. Esses arquivos permitem confrontar os números apresentados com as evidências da execução.

A auditoria é uma verificação de consistência e integridade dos registros; ela não representa uma assinatura independente de sua origem. Os dados completos e os scripts permanecem no repositório. Vamos voltar aos slides para discutir as limitações com Iasmin e Silas.

Slide 10 - Limitações e relação com inferência

Janela 12:55-14:15 | Duração-alvo 01:20



EM TELA - não ler em voz alta

Identificar a troca de Iasmin para Silas sem mudar o slide. Não apresentar hipóteses de frequência ou temperatura como causas medidas.

Próxima ação

Passar para Bruno e exibir o slide 11.

10/11

FALA - Iasmin Victoria Teixeira Barreto (12:55-13:35)

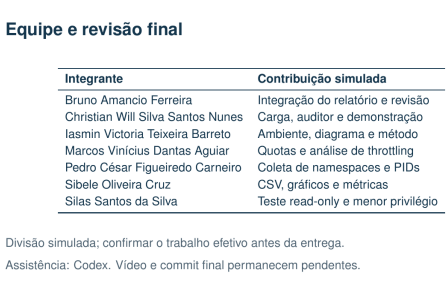
A principal limitação é que o host era compartilhado. Temos três repetições por condição e uma ordem alternada fixa, mas não controlamos continuamente carga externa, frequência ou temperatura. Esses fatores podem influenciar a variação e não foram isolados. Também houve uma limitação de visibilidade: o PID informado pelo Docker não apareceu no proc acessível via WSL. Assim, não confirmamos os limites ancestrais externos. Não substituímos essa ausência por uma afirmação de que eles estavam ilimitados.

FALA - Silas Santos da Silva (13:35-14:15)

A relação com inferência é conceitual. Limitar CPU, memória e tarefas ajuda a reduzir o impacto de uma carga em infraestrutura compartilhada. Um corpus somente leitura reduz alterações acidentais ou indevidas nos dados. Mesmo assim, kernel, runtime e host continuam sendo recursos e pontos de risco comuns. Nossa carga é aritmética Python: não carregamos um modelo de linguagem nem avaliamos GPU ou VRAM. Os resultados mostram os mecanismos observados neste laboratório, sem quantificar um serviço real de inferência. Bruno fará o encerramento.

Slide 11 - Equipe e revisão final

Janela 14:15-15:00 | Duração-alvo 00:45



Equipe e revisão final

Integrante	Contribuição simulada
Bruno Amancio Ferreira	Integração do relatório e revisão
Christian Will Silva Santos Nunes	Carga, auditor e demonstração
Iasmin Victoria Teixeira Barreto	Ambiente, diagrama e método
Marcos Vinícius Dantas Aguiar	Quotas e análise de throttling
Pedro César Figueiredo Carneiro	Coleta de namespaces e PIDs
Sibele Oliveira Cruz	CSV, gráficos e métricas
Silas Santos da Silva	Teste read-only e menor privilégio

Divisão simulada; confirmar o trabalho efetivo antes da entrega.

Assistência: Codex. Vídeo e commit final permanecem pendentes.

11/11

EM TELA - não ler em voz alta

Bruno fecha o vídeo. O slide 11 contém uma distribuição simulada de contribuições e pendências: conferir as atribuições com o trabalho efetivo antes da gravação final. Conferir se todos apareceram identificados em seus trechos.

Próxima ação

Encerrar a gravação. Conferir áudio, legibilidade e participação antes de publicar.

FALA - Bruno Amancio Ferreira (14:15-15:00)

Encerramos com três evidências centrais: namespaces distintos entre os contêineres, limites locais de cgroup confirmados e escrita negada no mount somente leitura. Na comparação de trabalho fixo, elevar a quota de meia para uma CPU reduziu o tempo mediano observado, dentro das limitações apresentadas.

Os scripts, os registros brutos, o gráfico e o relatório estão organizados no repositório. Também declaramos a assistência do Codex e a necessidade de revisão crítica pela equipe. A equipe permanece responsável por revisar os resultados e explicar os mecanismos utilizados. Obrigado por acompanhar nosso laboratório de Sistemas Operacionais.

Operação da demonstração

Apoio ao slide 9. Esta página orienta Christian e a pessoa que controla a tela.

Preparar antes de apresentar ou gravar

Abrir o repositório na sua raiz. Usar um ambiente em que python execute a versão configurada. No computador desta equipe, os ensaios anteriores usaram o ambiente Python no Ubuntu WSL.

Conferir o comando antes do ensaio e deixar o terminal com fonte grande. Não instalar programas, baixar imagem ou executar novamente a matriz durante a apresentação.

Deixar dois arquivos já abertos no editor: runs/C1-1.json e mount-readonly.json, nos caminhos completos abaixo. Esses arquivos registram observações anteriores; não são eventos que estão acontecendo agora.

Comando completo em uma única linha

```
python atividade3/scripts/audit.py atividade3/results/raw/20261108T013949Z-19921f7f
```

No Bash, a quebra de linha do slide usa uma barra invertida. A versão acima pode ser colada como um único comando. O auditor só verifica o lote existente; não precisa recriar os contêineres.

O que apontar na tela

1. Saída do auditor

Apontar valid=true, main_runs=6 e verified_files=68. Se a execução atual falhar, explicar o erro e abrir os registros preservados; não afirmar sucesso atual.

2. Registro C1-1

Apontar iterations=20000000, checksum=2589261568, correct=true, cpu.max="50000 100000" e cpu_stat_delta. Em C1-1, nr_throttled aumentou 71 e throttled_usec aumentou 3544785.

3. Mount read-only

Apontar content, write_denied=true, errno=30, uid=65534 e os controles de privilégio. Usar esse arquivo principalmente no vídeo; na apresentação, respeitar o minuto reservado à demonstração.

Arquivos para deixar abertos

`atividade3/results/raw/20261108T013949Z-19921f7f/runs/C1-1.json`

`atividade3/results/raw/20261108T013949Z-19921f7f/mount-readonly.json`

Plano de contingência: se o terminal não responder, mostrar os arquivos locais e explicar que são registros da coleta. Na apresentação, passar ao slide 10 sem consumir o tempo com diagnóstico. No vídeo, interromper o trecho, corrigir o ambiente e gravar novamente, preservando a honestidade sobre o que foi executado.

Preparação da gravação

Checklist de operação. A duração de 15 minutos é uma proposta deste roteiro, a ajustar no ensaio.

Identificação de todos

Gravar cada integrante com seu rosto visível ao menos em sua entrada, seu nome completo na tela e áudio reconhecível. Os nomes estão no quadro da primeira página. Não presumir que apenas a narração sem identificação demonstra participação.

Trocas de locutor

Nos slides 7 e 10, manter o slide durante a troca e identificar a pessoa que assume. Registrar pausas curtas para facilitar a edição, sem cortar explicações necessárias.

Slides e legibilidade

Compartilhar os slides em tela cheia e desligar notificações. Ampliar o terminal e o editor no slide 9. O público deve conseguir ler os campos que sustentam a explicação.

Falas e ritmo

Usar as falas como base, com compreensão e linguagem natural. Ensaiar o tempo total. Os números longos de handles e checksum podem ser apontados na tela; não é necessário soletrar todos os dígitos.

Slide 11 antes da gravação final

A tabela apresenta uma distribuição simulada de contribuições, detalhada em docs/contribuicoes.md. Conferir as atribuições com o trabalho efetivo de cada integrante antes da gravação final. A divisão simulada e as falas deste roteiro não comprovam autoria individual dos arquivos.

Demonstração honesta

Apresentar o auditor como verificação de arquivos existentes. O teste de escrita do JSON é uma evidência da coleta anterior. Não narrar essas observações como se os contêineres estivessem sendo executados naquele instante.

Revisão e publicação

Assistir ao vídeo completo, conferir o áudio de todos, os dados e a duração. Hospedar com acesso adequado ao docente. Inserir a URL no README, em VIDEO.md e na macro urlvideo do relatório LaTeX, recompilando o PDF.

Apresentação e entrega

O vídeo não substitui a apresentação de 11/11/2026. Preparar a entrega até 10/11/2026. No Classroom, disponibilizar o link do vídeo conforme o enunciado; guardar o comprovante de submissão.